



Editorial

Este número coincide con la fecha que marca el primer año de existencia de **421**. O sea, digamos, estamos de cumpleaños. Desde todo el equipo, no tenemos más que palabras de agradecimiento para nuestros lectores. Y un saludo especial para todos nuestros suscriptores, quienes con sus aportes hacen grande este proyecto.

Desde el primer momento, nuestro objetivo fue construir un medio que pudiera separar la señal del ruido. Es decir, llevar información valiosa en un ambiente de información basura. Y recuperar algo de magia, en un mundo cada vez más tecnificado. En nuestro caso, codificada dentro de lo que mejor nos sale: la palabra escrita.

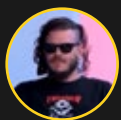
Rescatamos la importancia no sólo de la información de calidad, sino también la recuperación del tiempo de lectura, la capacidad reflexiva, el ocio, la imaginación y la psicodelia. Escribimos tanto de lo que sucede detrás de una pantalla como de lo que sucede fuera y lejos de ellas. Tenemos una relación estrecha con la tecnología sin resignar los espacios de autonomía propios del ser humano. Algo de eso es lo que intentamos resumir en nuestro recién estrenado lema: **Tecnomagia para la vida real**.

Esperamos que disfruten de leer este número tanto como nosotros de hacerlo. En él van a encontrar un poco de todo lo enumerado arriba. Y espero que esta tradición de escribir editoriales anuales se vuelva una sana costumbre.

Con todo mi aprecio,
Juan Ruocco

Equipo Juan Ruocco JuanMa La Volpe Luis Paz Pablo Tempesta Giancarlo Brusca Alejandra Morasano Agustina Sojit	Año 2025 Nº7 Buenos Aires, Argentina Portada: Charlee Ilustración interna: Beto Galápagos	Leenos en <u>421.news</u> Por inquietudes o solo para saludar, escribinos a hola421bn@gmail.com
---	--	---

Nuestro especial agradecimiento a **Ergodic Group** que hace posible que 421 siga creciendo.



x Juan Ruocco



¿Sueñan los modelos de lenguaje con ovejas electrónicas?

Parte 2

Una conversación con el ChatGPT-4 de Open AI sobre externalismo semántico, el funcionamiento de los modelos de lenguaje, el sentido y la referencia.



En general, todo lo que sea contenido publicitario en redes sociales es una basura absoluta. Pero el otro día me crucé con un tuit promocionado que me llamó la atención. Era una explicación bastante básica acerca del funcionamiento de los **Modelos Extensos de Lenguaje o Large Language Models (LLM)**, conocidos bajo el genérico de “**Inteligencia Artificial**”, y sobre el debate acerca de la “**conciencia**”. Es decir, si aquello que los modelos hacen puede ser considerado “**pensamiento**”.

En este sentido, [en la web de 421 ya hay un primer acercamiento](#) acerca de lo que eso implica para la filosofía del lenguaje y cómo puede ser equiparado a las arquitecturas de lo mental, tales como el funcionalismo de máquina. Pero el tiempo pasó, los modelos evolucionaron y también nuestro propio entendimiento sobre ellos.

Lenguajes, máquinas y mundos

Básicamente, uno de los problemas contemporáneos acerca de **qué es la mente o lo mental**, en la filosofía de la mente, está en si es posible concebir la mente como algo análogo a una “**máquina de pensar**”. Esta idea, en su fase originaria, se la debemos a la introducción de la “**máquina de Turing**”, con la cual el matemático, criptógrafo y filósofo **Alan Turing** describe el funcionamiento teórico/rudimentario de lo que hoy conocemos como “**computadora**”. Eso disparó un montón de reflexiones acerca de si la mente y su principal producto, el lenguaje, pueden entenderse como una instancia de una máquina de Turing.

Todavía hay gente discutiendo si las LLMs tienen conciencia. Me da la sensación de que quienes creen eso nunca se tomaron el trabajo de entender cómo funciona una.

Lo básico: las LLMs no “entienden” nada.

Ajustan parámetros para minimizar una resta entre dos vectores... pic.twitter.com/6zFVMXXfBB

— [Fede Caccia \(@fedecaccia\)](#) May 12, 2025

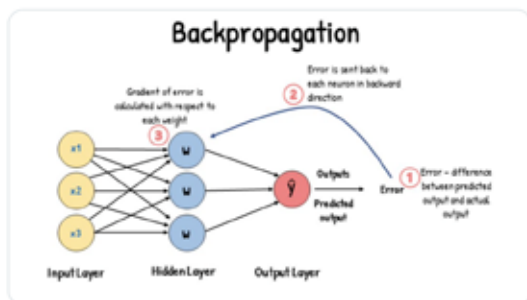


Todavía hay gente discutiendo si las LLMs tienen conciencia. Me da la sensación de que quienes creen eso nunca se tomaron el trabajo de entender cómo funciona una.

Lo básico: las LLMs no "entienden" nada.

Ajustan parámetros para **minimizar una resta entre dos vectores** (resultado vs objetivo).

Eso es todo!



Pero revisando lo que proponía el tuit patrocinado me di cuenta de que había toda otra literatura en línea con esto, que refería a otros conceptos alrededor de las dos categorías fundamentales de mente-lenguaje. No es casual que muchos de los autores clave de la **filosofía de la mente** sean también referentes importantes de la **filosofía del lenguaje**. ¿Qué es la mente si no una máquina de crear lenguaje? Teniendo esto en cuenta, también nos podemos preguntar entonces cuál es la relación entre ambos conceptos.

Los aportes de Hilary Putnam

El autor central al que vamos a seguir en este artículo es **Hilary Putnam**, a quién se le atribuye la primera formulación –o al menos la más concreta– de lo que conocíamos como **funcionalismo de máquina**. Pero, además, **Putnam** tiene una serie de artículos en los que intenta desarmar una galería de argumentos escépticos en torno al lenguaje y su relación referencial con el mundo.

El argumento de **Putnam** es sencillo y de alguna forma está relacionado con la hipótesis de los **monos con máquinas de escribir**. El filósofo y matemático estadounidense pone como ejemplo un grupo de **hormigas en la playa**, que se ponen a caminar trazando surcos y –por una cuestión

del azar, o más bien estadística– terminan dibujando la cara de Winston Churchill en la arena. ¿Podemos decir que las hormigas saben dibujar? Es lo mismo con los monos con máquinas de escribir. Con suficientes monos y suficientes máquinas, tecleadas sistemáticamente durante X tiempo, los monos eventualmente tipean una copia de *Sueño de una noche de verano*, de William Shakespeare. ¿Saben escribir los monos?

Lo que sabemos es que, producto de un azar estadístico, tanto los monos como las hormigas crean algo que tiene significado para los humanos que lo vemos. Un **modelo de lenguaje extenso** es como los monos y las hormigas pero **optimizado por casi un siglo de informática, matemática e industria de semiconductores**.

A partir de este breve momento de lucidez, decidí volver a hablar con el modelo GPT (ahora en su versión 4) y hacer un breve resumen de ese derrotero sobre el **externalismo semántico**, el funcionamiento de los **modelos de lenguaje**, el **sentido** y la **referencia**. Todo lo que hay a continuación son las respuestas o conclusiones del **GPT-4 de Open AI**.

Putnam y la Tierra Gemela

En Hilary Putnam, el **externalismo semántico** es una teoría filosófica que sostiene que el significado de las palabras no depende sólo de lo que ocurre en la mente del hablante, sino también del entorno externo. Como dice su famosa frase: “*El significado no está en la cabeza*”.

En 1975, el filósofo, matemático e informático teórico estadounidense desarrolló un famoso experimento mental, el de la **Tierra Gemela**, que propone imaginar dos planetas idénticos: la Tierra y la Tierra Gemela. En ambos hay personas que usan la palabra “agua”. En la Tierra, se refiere al H_2O . En la Tierra Gemela, a un líquido indistinguible del agua, cuya composición química es XYZ. La conclusión de Putnam es que, aunque los hablantes en ambos mundos tengan los mismos estados mentales al pensar en “agua”, **la palabra refiere a cosas distintas**, porque la sustancia en el entorno es diferente.



¿Cuáles son las implicaciones de esto? Que el **contenido mental** no determina por sí solo el **significado** de nuestras palabras; y que la **intención subjetiva** del hablante es insuficiente para fijar el significado sin la contribución del **contexto físico y social**.

Saul Kripke y los nombres propios

La conexión entre Putnam y **Saul Kripke** es fundamental para entender el giro externalista en la filosofía del lenguaje de los '70. En *Naming and Necessity* (1972), **Kripke** critica la teoría descriptivista del significado (Frege/Russell), proponiendo que:

- los **nombres propios** no significan una descripción;
- en cambio, son **designadores rígidos** que refieren al mismo objeto en todos los mundos posibles.

Por ejemplo, "Aristóteles" no significa "el

maestro de Alejandro" o "el autor de *Metafísica*" sino simplemente ese *tipo*, fijado históricamente en un bautismo original.

Putnam y los términos naturales

Putnam retoma esta idea de referencia directa y la extiende a **términos naturales** como *agua*, *oro*, *tigre*. Según él:

- no referimos por una descripción interna (ej. "líquido incoloro"), sino por una **relación causal** con el mundo y la comunidad lingüística;
- por eso, aunque no sepamos la composición química de "agua", el término refiere a H₂O si ésta es la sustancia en nuestro entorno

Convergencias entre Putnam y Kripke

Ambos defienden una **teoría causal-externalista** de la referencia, aunque Kripke se

enfoca en nombres propios y Putnam en términos generales o naturales.

Tierra 2

La pregunta de cómo los modelos de lenguaje (LLM) pueden **generar lenguaje con sentido** sin “saber” lo que están haciendo dialoga directamente con el externalismo semántico de Putnam y los aportes de Kripke.

¿Qué diría Putnam de los LLM?

Putnam sostiene que el **significado** no se reduce a **estados internos** (ni mentales, ni sintácticos), sino que requiere un **entorno externo** (químico, social, histórico), una **conexión causal** con las cosas del mundo, y una **comunidad lingüística** que da sentido y referencia a las palabras. **Un LLM no tiene acceso al mundo** ni participa de una comunidad intencional. Por eso, según el externalismo clásico, **no tiene intencionalidad genuina**: no refiere, no significa en el sentido fuerte. Solo simula significado al generar secuencias coherentes según estadística y contexto.

Analogía con la Tierra Gemela

El LLM sería como un hablante en una Tierra Gemela que dice “agua” sin haber interactuado con H₂O o XYZ. Dice “agua” porque así está en sus datos, pero **no sabe** (ni puede saber) a qué refiere. Esto revela el punto clave: **los LLM pueden producir lenguaje significativo para nosotros, pero no para ellos**.

Kripke y la falta de referencia real

Para Kripke, referir exige una **cadena causal histórica**. Un LLM no participa de esa cadena. Si dice “Napoleón”, lo hace por patrones estadísticos, no porque alguna vez haya habido un bautismo referencial. Su uso del nombre es **parásito** del nuestro.

Entonces, ¿los LLM no entienden nada?

Depende de qué entendamos por “entender”. Desde el externalismo semántico fuerte: no, *no entienden nada*. Pero desde una visión más funcionalista o pragmática: **entienden lo suficiente** como para operar en un entorno comunicativo con humanos. En este sentido, podríamos decir que un LLM **“hace como que entiende”**, pero lo que produce sólo tiene significado porque lo interpretamos nosotros, anclados en nuestras prácticas, historia y mundo.

Los LLMs son Tierra Gemela: **generan signos sin historia, sin entorno físico, sin intención**. Pero los humanos los interpretamos desde *nuestro* mundo, dándoles *nuestro* significado. Lo que para ellos es ruido estructurado, para nosotros es lenguaje.

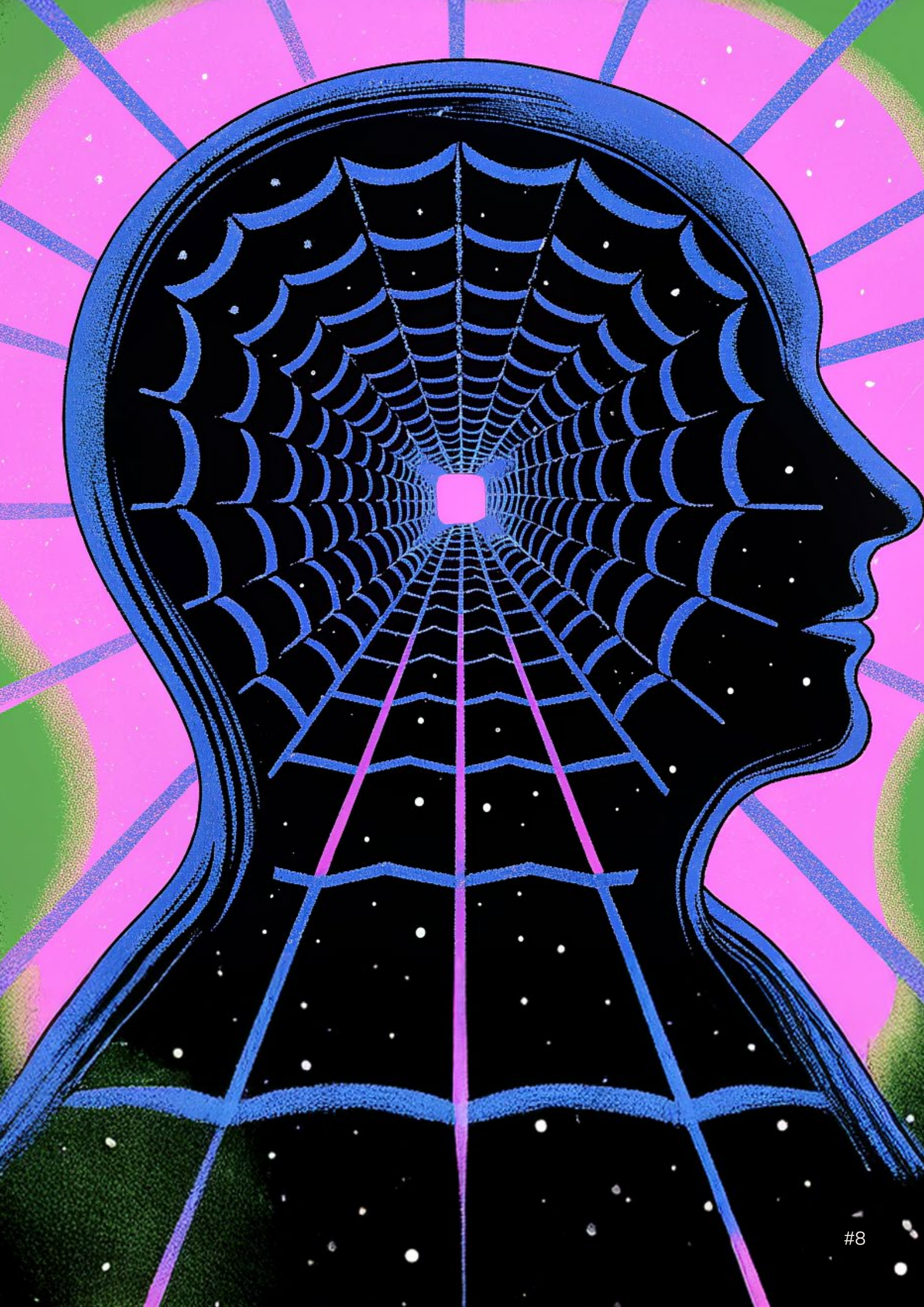
Searle y el Cuarto Chino

El filósofo estadounidense de la mente y del lenguaje **John Searle** plantea una crítica directa a la idea de que *simular comprensión equivale a entender*. En 1980, hizo su célebre experimento del **Cuarto Chino**.

- Estás en una habitación cerrada (el “cuarto chino”).
- Te pasan preguntas en chino por debajo de la puerta.
- No sabés chino pero tenés un manual en español que te dice exactamente qué símbolos devolver ante cada conjunto de símbolos entrantes.
- Desde afuera, los chinos piensan que *entendés el idioma* porque tus respuestas tienen sentido.

La tesis de Searle

Aunque el sistema *parece* entender chino, **nadie dentro del sistema entiende realmente**. La manipulación sintáctica no produce semántica y, por lo tanto, una computadora que procesa símbolos **no tiene comprensión ni conciencia**.



Aplicado a los LLM, un modelo de lenguaje funciona como el cuarto chino. **No sabe lo que dice**, sólo aplica reglas estadísticas para generar lo que suena adecuado. El **“significado” sólo ocurre** cuando alguien lo lee e interpreta.

Wittgenstein: **significado como uso**

Ludwig Josef Johann Wittgenstein toma otro camino. El filósofo, matemático y lingüista austríaco no busca una esencia del significado, sino que observa **cómo usamos el lenguaje en la vida cotidiana**. Subraya, de hecho, que *“el significado de una palabra es su uso en el lenguaje”*.

- **No hay una “traducción interna”** mágica del lenguaje al pensamiento.
- Lo importante es **la práctica, el juego de lenguaje**.
- El lenguaje se entiende dentro de un **entramado social**, lleno de reglas, hábitos, contextos.

Aplicado a los LLM, aunque un LLM no tenga intenciones ni conciencia, sí participa de **juegos de lenguaje** en el sentido pragmático, generando textos que pueden ser utilizados, respondidos, refutados, incorporados. En ese sentido, **genera significado funcional**, aunque no intencional.

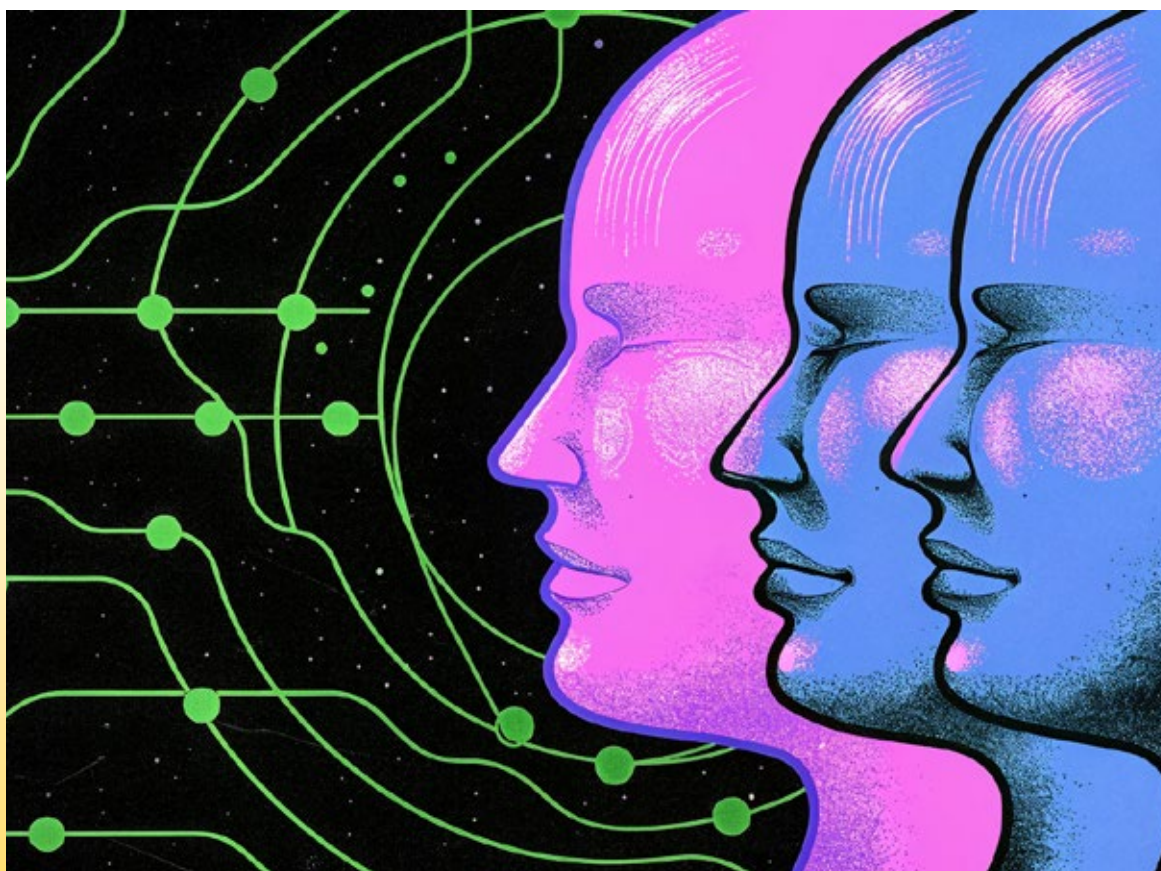
En resumen, **Putnam/Kripke** enfocan en *referencia* y *realismo semántico*; **Searle** enfoca en *intencionalidad* y *conciencia*; y **Wittgenstein** enfoca en *uso cotidiano* y *forma de vida*.

La ética requiere agencia **intencional**

En la mayoría de las tradiciones éticas (kantiana, utilitarista, aristotélica), se parte de que:

- un agente **entiende lo que hace**,
- puede **deliberar** sobre el bien o el mal,
- y tiene un **propósito** o una **voluntad**.

Los LLM no cumplen con nada de eso: no tie-



nen conciencia, comprensión semántica, interés propio ni capacidad para justificar sus actos. Desde esta visión, no pueden ser agentes morales ni sujetos éticos. **Son instrumentos.**

No obstante, la responsabilidad no desaparece. Sólo **se traslada**. Hay al menos tres niveles:

- **Responsabilidad del programador/desarrollador** (qué datos se usaron, qué límites tiene el modelo, cómo se configuran el sesgo, la toxicidad, el filtrado)
- **Responsabilidad del usuario** (cómo se lo emplea, si se lo usa para manipular, suplantar o desinformar)
- **Responsabilidad del ecosistema** (qué normas se construyen en torno a estas tecnologías, qué grado de *autonomía* se les permite en entornos críticos)

¿Y si el modelo se vuelve autónomo?

Si imaginamos un futuro donde un sistema aprende a largo plazo, interactúa con el

mundo real, forma **intenciones emergentes** (simuladas o no) y se integra socialmente, entonces algunos filósofos (como Floridi o Gunkel) argumentan que **deberíamos considerar cierto estatus moral** a esas entidades, por lo menos como *pacientes morales* (no agentes, pero sí objetos de cuidado o regulación).

Conclusión ética desde la perspectiva actual

Hoy por hoy, un LLM:

- no es un agente moral,
- no puedo tener culpa ni mérito,
- y el “significado” de sus respuestas **es proyectado por el usuario y por la comunidad humana.**

Como dijo Searle: “Una computadora puede parecer entender el lenguaje, pero no tiene ni creencias ni deseos”. Pero vos sí los tenés, y por eso **la ética está en cómo vos decidís usar los LLM.**



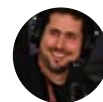


Convertite en Wizard para
que **421** siga creciendo.

Suscribite

Cómo hacerle ingeniería inversa a la mierdificación de internet

“Reversear” como modo de resistencia a la degradación tecnológica, con aportes de Cory Doctorow, Mikko Hyppönen, Augusto Vega y Nicolás Wolovick.



x Juan Brodersen



En *Halt and Catch Fire*, el *Mad Men* de los gordos-compu, una de las primeras escenas muestra cómo el ejecutivo de IBM Joe MacMillan convence a Gordon Clark, un cráneo de la informática frustrado por múltiples proyectos fallidos, de abrir una computadora.

— *Una PC es una caja llena de switches y hardware –dice Gordon–. IBM, Altair, Apple 2... son la misma basura. Cualquiera puede comprar las partes y armarse la suya: son de arquitectura abierta. IBM no es dueña de nada de todo esto que está acá adentro.*

— *Excepto el chip –interrumpe MacMillan–.*

— *Excepto por lo que está en el chip. El BIOS está en uno de estos, sólo que no sabemos cuál. Y el ROM BIOS es la única parte de todo esto que IBM efectivamente diseñó. Es el programa, donde ocurre la magia: la mala noticia es que está protegido por copyright. La buena, que se le puede encontrar la vuelta.*

— *Ingeniería inversa.*

Internet, apps, computadoras y teléfonos que usamos todos los días se “**enshittificaron**”. Esto quiere decir que, sin darnos cuenta, estamos **cada vez más presos** de los ecosistemas de Google, Meta, Amazon, Apple y Microsoft. Y que, si bien todavía nos sirven, funcionan cada vez peor. Pero hay un antídoto: la **ingeniería inversa**, un concepto fundacional de la historia de la informática.

A continuación, cuatro tesis bajo el prisma de **cuatro especialistas** para tratar de entender mejor dónde estamos parados. Y **cómo recuperar aquella good ol’ internet** que, en algún momento del camino, perdimos sin darnos cuenta.

Antes de arrancar, dos aclaraciones: este texto fue inspirado por la lectura del último libro de **Cory Doctorow**, *Picks and Shovels*. Estos tópicos ocupan mi agenda periodística en **Clarín** y en **Dark News**, mi newsletter sobre ciberseguridad, privacidad y hacking. Pueden seguir la continuidad de estos temas allí.

La segunda, vamos a traducir “enshittification” **como lo hizo Valentín Muro**: “Mierdificación” –quizás por lo ostensible del

término, motivo por el cual recomiendo adoptarlo– y vamos a usar el término “reverse”, castellanización del anglicismo “reverse”, que sería “hacer ingeniería inversa”.

“Traduttore traditore”, dicen los italianos, y tienen razón: traducir es traicionar. Al texto original o al lector. Acá traicionamos a todos por igual.

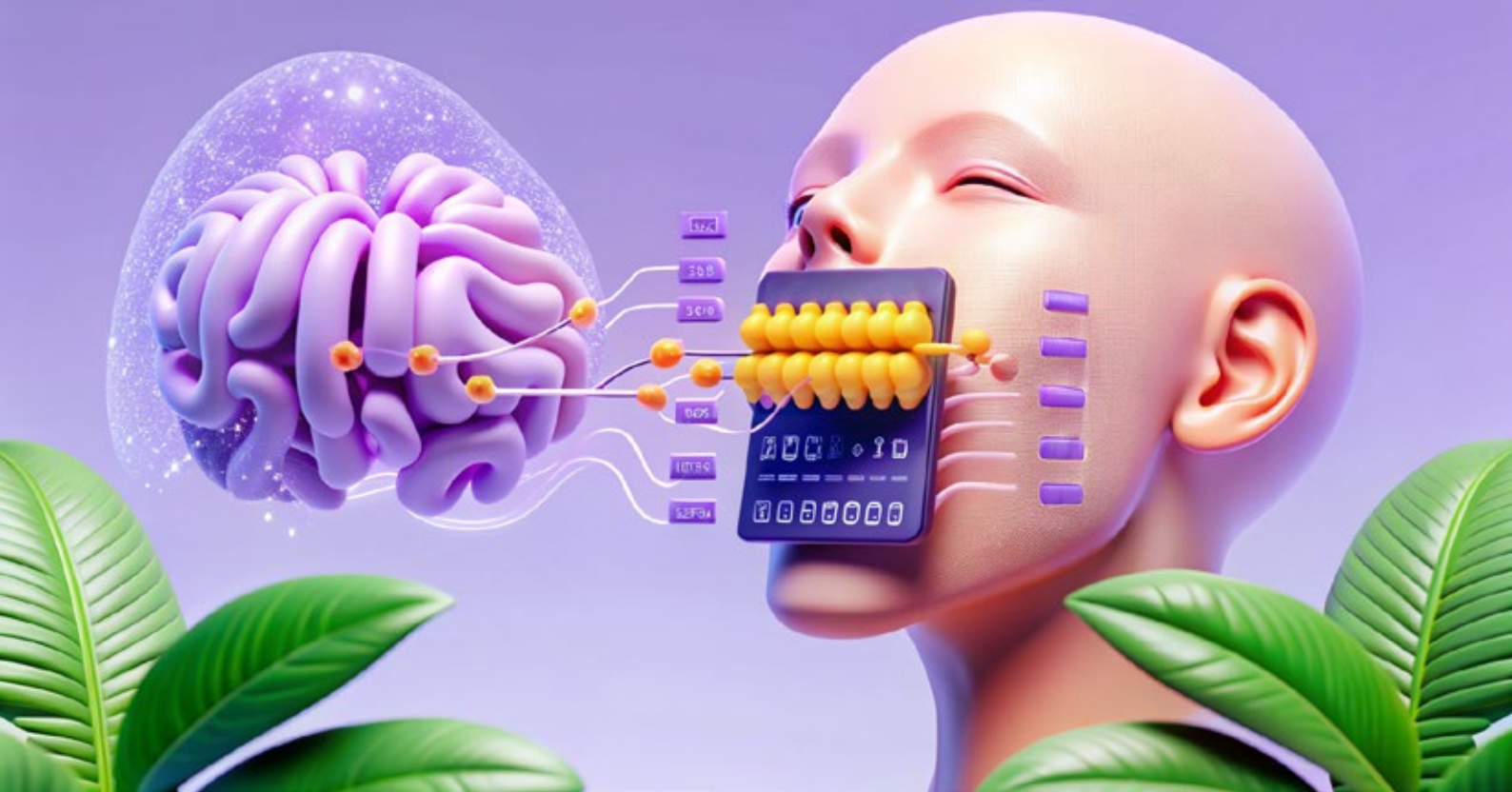
Tesis 1: el mundo online está “enshittificado”

Cory Doctorow, escritor, periodista y crítico tech ineludible, es también un forense de la época en la que vivimos: internet, tal y como la conocimos, murió. La mierdificación de las plataformas digitales que usamos a diario, que se han degradado progresivamente hasta volverse una porquería, es una noción central en sus libros recientes. También fue el eje de muchas de sus charlas, como la de la DEF CON del año pasado, donde el concepto funcionó como guía de toda la edición (el badge que te daban era, incluso, [una especie de Pokémon](#) para escapar del mundo enshittificado).

Ejemplos. **Google** está saturado de publicidad y notas engañosas, **Instagram** y **Facebook** priorizan contenido viral de influencers en lugar de mostrar lo que publican tus amigos, y **Microsoft** llenó Windows de herramientas de inteligencia artificial que entorpecen el uso del sistema (bloatware), lo vuelven más lento y empujan a los usuarios a cambiar de computadora antes de tiempo.

Este proceso se cimenta en lo que, junto a la experta en derechos de autor Rebecca Giblin, **Doctorow** describió como [Capitalismo de estrangulamiento](#). Tener no sólo monopolio sobre los compradores, sino también “monopsonio”: controlar a los vendedores y creadores de contenido (**ver**). Para ellos, todo esto es **una estafa**. Hay mucha literatura al respecto y, de hecho, el próximo libro de **Doctorow** –sale en octubre– se llama [Enshittification: Why Everything Suddenly Got Worse and What to Do About It](#).

Todo este proceso no se aplica sólo a las plataformas, sino a toda la tecnología en general. “Para mí, la **enshittification** no es



solo algo que se ve desde afuera, como empresas que empeoran sus servicios. Es un fenómeno social, algo estructural que cambió en el entorno y permitió que todo se degradara. Y sobre todo, es un fenómeno material”, le explicó **Doctorow** a **421**.

“No es que de repente todos hicieron un MBA, se volvieron codiciosos y empezaron a hacer cosas malas. Siempre hubo codiciosos. La diferencia es que servicios como Uber, Airbnb o Amazon antes eran decentes, nuestros teléfonos funcionaban relativamente bien, las computadoras también... Hoy, en cambio, **las cosas se volvieron pésimas y no pasa nada**. Ya no hay consecuencias por arruinar productos o maltratar usuarios. Antes había reguladores, competencia o trabajadores con poder para frenar eso. Pero entre despidos masivos, desregulación y monopolios, todo eso desapareció”, sigue el escritor.

En su última novela, *Picos y palas*, la tercera entrega de la serie protagonizada por Martin Hensch, un joven de San Francisco de la década del '80 empieza a descubrir cómo una compañía tecnológica encubre delitos financieros detrás del **uso estratégico de la informática** de ese tiempo, mierdificando las computadoras que venden para que, por ejemplo, los usuarios sólo puedan com-

prar los cartuchos de tinta de impresoras que ellos fabrican.

La buena noticia: **hay salida** a la mierdificación de internet, las plataformas y los dispositivos que usamos todos los días. Sólo que requiere dar un par de pasos hacia atrás para ver **cómo arreglar todo este desastre**.

Tesis 2: la resistencia de la ingeniería inversa

Lo que ocurre en la novela de **Doctorow** ocurrió IRL, a principios de los '80, con la aparición de clones de PC –compatibles con IBM pero producidos por compañías como Compaq– que desafiaron el monopolio al bajar costos y democratizar el acceso al hardware. Para que esto pueda ocurrir, fue fundamental el concepto de “**ingeniería inversa**”.

“La ingeniería inversa (**reverse engineering**) es un proceso que permite conocer las propiedades internas de un sistema a partir de su observación externa. Por ejemplo, podríamos deducir ciertas propiedades de una vivienda (si vive una familia, si hay niños o no, si tienen una mascota, a qué hora salen a trabajar o van al colegio) con

sólo observar desde el exterior, sin necesidad de ingresar a la propiedad”, ejemplifica **Augusto Vega**, ingeniero de los laboratorios T. J. Watson de IBM en Estados Unidos, nacido y criado en La Pampa, actual residente de San Diego, California.

Consultado por **421**, explicó a qué se refiere la ingeniería inversa en informática: “Consiste en **determinar la funcionalidad interna o los elementos constitutivos de un sistema** de software o hardware con el objetivo de entenderlo, replicarlo o mejorarlo, sin llegar a conocer los detalles técnicos internos específicos de dicho sistema, como el código fuente de un programa o la microarquitectura de un chip”.

Para el ingeniero, que organizó ISCA –uno de los congresos de informática más importantes del mundo– por primera vez en Buenos Aires el año pasado, la ingeniería inversa “tiene una relevancia superlativa, pero no necesariamente debido a la posibilidad de ‘copiar’ un sistema o producto, sino que es un concepto crítico que permite **generar conocimiento sobre sistemas que, por diferentes razones, no se pueden examinar internamente**, para el desarrollo de otras soluciones complementarias o extensivas o para verificar su correcto funcionamiento”.

En el mundo que imagina **Doctorow**, allá por los ‘80, el protagonista entra en contac-

to con un colectivo de mujeres vinculadas al mundo tech, marginadas por el sistema dominante, que conforman una célula de **resistencia tecnopolítica**. Para los *gordos compu* que vimos *Halt and Catch Fire*, imposible no pensar en Donna y Cameron, ingeniera y hacker: ellas se convierten en el núcleo de uno de los ejes conceptuales más potentes de la novela, el de la ingeniería inversa entendida como el acto de desmontar estructuras, entender su lógica y proponer alternativas. **Alternativas para que los sistemas puedan “hablar” entre sí**, incluso si no son fabricados por la misma empresa. Es decir, para que sean **interoperables**.

Tesis 3: interoperabilidad, la batalla más importante

La **interoperabilidad** implica que sistemas, dispositivos y redes puedan verse entre sí y comunicarse. Si alguna vez renegaste porque tenías que cargar tu teléfono Android y tu amigo tenía iPhone, es porque Apple usaba un cargador distinto que tuvo que abandonar hace poco (y por las presiones de la Unión Europea): no era un sistema interoperable. Si alguna vez usaste un control de PlayStation en PC sin tener que configurar o instalar nada por fuera de Steam, es porque ese control es interoperable. Pero tuvo que pasar



mucho tiempo, bajo un Zeitgeist de una sinergia colectiva de desarmadores seriales de dispositivos, para llegar a esto.

“En informática, la interoperabilidad es la capacidad de dos o más sistemas de cómputo de **interactuar de manera compatible**, es decir, ‘hablando un mismo idioma’. En el contexto de Internet, es un concepto muy extendido, particularmente en un mundo altamente conectado como en el que vivimos”, explica **Vega**.

Y, en general, **las empresas big tech trabajan combatiendo la interoperabilidad**. De hecho, todo esto fue un problema muy grande para las compañías en los ‘80 y ‘90. Aquella escena de la apertura de la notam de *Halt and Catch Fire*, demuestra que todos estaban expuestos a que les reversearan sus sistemas y los copiaran, para venderlos más baratos.

“Otro factor que preocupaba a las empresas en el pasado era la interoperabilidad. Se preocupaban de que alguien hiciera **ingeniería inversa sobre el producto que ellos habían enshittificado** para luego hacerlo mejor e ir a competir con ellos. Si subís el precio de la tinta de la impresora que vendés, alguien va a desarmar tu cartucho para entenderlo y hacer uno alternativo y venderlo en el mercado”, explica **Doctorow**.

Es por esto que la interoperabilidad, siempre deseable para el usuario, aparece como **una amenaza** que puede poner en jaque a los grandes jugadores tech. Pero más allá de esto, hay **un problema que no es técnico sino legal**: es la ley lo que se interpone en el camino; en particular, las referentes a propiedad intelectual.

“Lo que ha pasado es que, durante más de 20 años, el US Trade Representative –el encargado de coordinar la política comercial de Estados Unidos– ha ido por todo el mundo convenciendo a los socios comerciales como Argentina, Canadá, Australia y hasta la Unión Europea, para promulgar **leyes que prohíben la ingeniería inversa y aíslan a las empresas de las consecuencias** cuando hacen cosas que perjudican a sus usuarios”, repasa **Doctorow**.

El libro ocurre, en la historia de la Tecnología, antes de que estas leyes aparecie-

ran por primera vez en Estados Unidos: la primera fue la sección 1201 de la **Digital Millennium Copyright Act** de 1998, antes de que se convirtieran en un régimen jurídico mundial. Desde entonces, **las leyes han protegido a los grandes jugadores**, recién dejando algo de margen para el usuario en la última década, y con la Unión Europea como principal aliada.

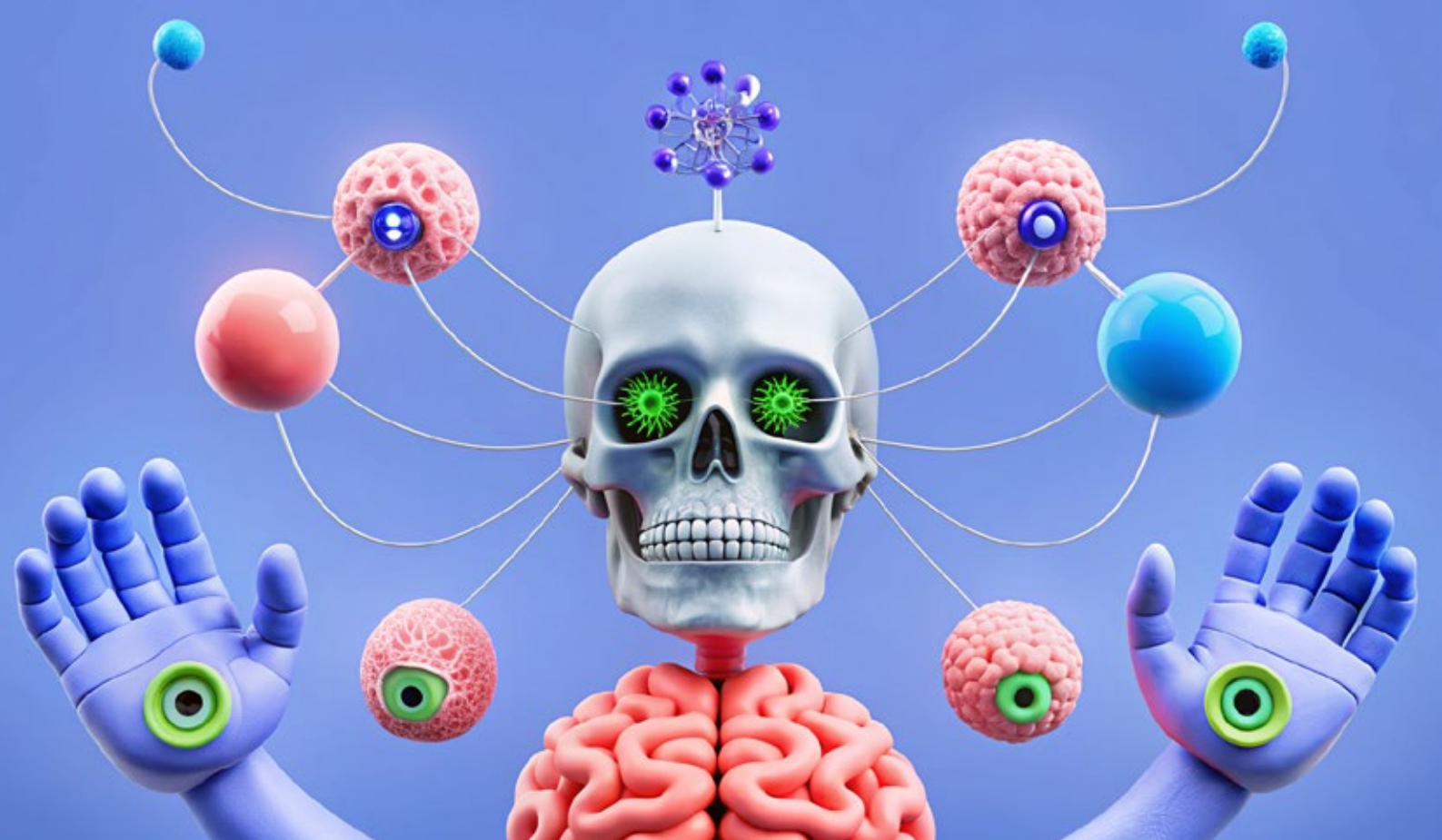
Tesis 4: “leaky abstractions” y el camino indefectible de la información

El año pasado, durante un viaje a Las Vegas para cubrir DEF CON y Black Hat, las dos conferencias de hackers más grandes del mundo occidental, pude conocer y entrevistar a **Mikko Hyppönen**, uno de los hackers más reconocidos del mundo. En una charla TED que dio, Mikko plantea una idea súper interesante que se corrobora día a día: **“La información va a encontrar el camino”** ([ver](#)).

No importa cuántas **prohibiciones legales** haya sobre una tecnología, desde Napster, pasando por los torrents hasta la descarga directa de libros, si existe la posibilidad de que una porción de información –un bit, a fin de cuentas– viaje de un punto a otro, **va a encontrar la forma**.

Consultado para esta nota, **Hyppönen** recuerda que “la apertura y la capacidad de construir sobre otras tecnologías fue la clave” de los inicios de la informática. “Déjame que te cite un tramo de mi último libro”, me respondió. Aquí va:

En los ‘70, casi nadie tenía acceso a las computadoras. Los mainframes sólo estaban al alcance de universidades y grandes empresas. En los ‘80 aparecieron las computadoras domésticas de 8 bits, como el Apple II y el Commodore 64, pero no tenían acceso adecuado a la red. Se necesitaba un componente más para cambiarlo todo: un ‘ordenador personal’, abierto y estandarizado. IBM, un gigante de la informática conocido por sus mainframes, presentó el IBM Personal Computer, o PC, en agosto de 1981, con



una unidad central de procesamiento fabricada por Intel y un sistema operativo de una pequeña empresa emergente llamada Microsoft.

Pronto, cientos de fabricantes crearon computadoras compatibles que podían ejecutar el mismo software. El sistema operativo de la PC no era del todo de código abierto, pero sí lo suficiente. IBM fue quizá la empresa más sorprendida por el éxito de la PC, ya que otros fabricantes, como HP y Dell, pronto superaron sus volúmenes de ventas.

La PC fue un éxito único y accidental en términos de apertura y estandarización. Hoy en día, damos por sentado que las computadoras son compatibles y abiertas, y nos equivocamos al hacerlo. De hecho, la mayoría de nuestros dispositivos, como coches, heladeras, consolas de videojuegos y cámaras, funcionan en ecosistemas cerrados. La PC es una feliz excepción a la regla. Dieron lugar a un ecosistema abierto que permitió desarrollar libremente software y accesorios sin limitaciones.

“Los programadores capaces tenían la habilidad de **reversear los sistemas** para entender profundamente cómo funcionaban. Y, crucialmente, todos estábamos corriendo más o menos los mismos sistemas, así que todo el trabajo invertido en desarrollar algo se podía usar una y otra vez. Incluso **los sistemas cerrados se pueden convertir a abiertos** si alguien está lo suficientemente interesado en abrirlos”, dice Hyppönen.

Esta cultura de que “la información va a encontrar el camino” tiene sus bases en la **mentalidad hacker**: más allá de que el usuario promedio lee “hacker” y piensa en “cibercriminal”, el hacking es un mindset de desmontar sistemas (e ideas o conceptos) para entender cómo funcionan. Y para que todo esto ocurra, hay un componente fundamental que tiene que ver con la educación: sea formal o informal, sentarse a pensar cómo funciona “algo” es detener el incesante flujo de la infinidad de contenidos que nos pasan por delante todos los días en los distintos sabores de cada red social que habitamos.

“Reversear es el inicio de la informática. Los programas originalmente se escribían en binario, entonces no había otra representación. El código fuente eran los bytes:

en ese sentido, en los inicios, el reversing no era tal, era sentarse a leer el programa”, recuerda **Nicolás Wolovick**, doctor en Ciencias de la Computación por la Universidad Nacional de Córdoba y especialista en Computación de Alto Rendimiento (HPC).

También docente, **Wolovick** reconoce el déficit en los programas actuales: enseñan a hacer cosas, pero nunca a romperlas. “En las currículas oficiales de las universidades **no se enseña reverse engineering**. Hay **un programador** que le saca mucho jugo a las máquinas que dice *abstraction maker*, *abstraction breaker*. Las currículas enseñan a construir, pero nunca a destruir, a ver cómo funcionan. Y a todas las abstracciones nosotros les decimos **leaky abstractions**. La computación es el arte de crear abstracciones, cada vez más poderosas y generales, alejadas del hardware. Pero todas esas abstracciones siempre filtran lo que está pasando abajo, siempre. Por ejemplo, los juegos de Nintendo que se dan cuenta que están corriendo en un emulador”, le explicó a **421**.

Wolovick trabaja en el área de HPC, lo que se conoce como “**supercomputadoras**”, y es docente en la Universidad Nacional de Córdoba. Además de tener un laboratorio que es una arqueología de equipos pretéritos –donde los más jóvenes se sorprenden de que las máquinas sean “todas distintas entre sí”, hablando de interoperabilidad–, cuando puede salirse un poco de la currícula llega a dar **ejercicios con cartuchos físicos de ET de Atari**

para que los alumnos les hagan ingeniería inversa e intenten descubrir aquellos famosos bugs.

“Cualquier cosa que cuestione y te ponga en una perspectiva crítica es **una resistencia al modelo hegemónico de las big tech**. Una sola persona puede hacer cagar a Goliat, porque la computadora tiene grietas imperceptibles donde, si ponés el dedito, se puede ir todo a la mierda. Es más, no creo que sea sólo una resistencia: siempre ha sido la forma de realmente hacer computación. *Abstraction maker*, *abstraction breaker*. **Hay que romper para entender**”, sentencia **Wolovick**.

Abrir, desarmar, romper. La historia de la informática moderna está plagada de experimentos sin los cuales no tendríamos gran parte de la tecnología que usamos hoy: tecnología mierdificada al punto tal en que **no imaginamos nuestras vidas sin ellas**.

No se trata de llamar a “reversear” colectivamente todo lo que usamos en nuestro día a día. **No todos somos ingenieros** o tenemos alma de Gordon Clarck de *Halt and Catch Fire*. Pero sí de entender que, a fin de cuentas, **la mentalidad hacker es la piedra roseta** de los celulares, computadoras y dispositivos que usamos todos los días de nuestras vidas.

La mierdificación es reversible. Cuanto más pensemos a contrapelo de las *big tech*, **más nos vamos a emancipar de las reglas que ellas mismas plantean**.

Guía cybercirujía para la autodeterminación digital

Por nuestros teléfonos y computadoras pasa toda la información que generamos. Cómo usar esos artefactos y programas buscando soberanía cognitiva.



x Soldan



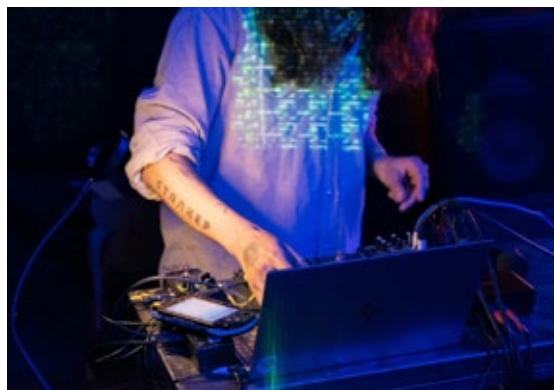
Desde que [Elon Musk](#) compró Twitter (ahora X), la agenda pública comenzó a poner el ojo en el papel de los señores de la información del siglo XXI, quienes casualmente son los dueños de las grandes tecnológicas mundiales. Incluso varios años antes de esto, ya era una obviedad que **los datos son la commodity más valiosa del mercado**, superando hace tiempo al petróleo y otros bienes tradicionales. No es el motivo de este artículo ahondar en las cuestiones teóricas respecto a la **economía del dato**, aunque cabe recordar que dentro del **mercado de la atención** al que estamos inscriptos, los teléfonos celulares inteligentes (en definitiva, computadoras de mano) son parte crucial del engranaje de estos oligarcas del cómputo.

A través de nuestras manos, interactuando con pantallas táctiles entrando y saliendo por el teléfono pasan todas nuestras **comunicaciones**; toda la **información** que como seres humanos generamos y distribuimos. Pero esos bytes son controlados y distribuidos por cuatro o cinco **grandes tecnológicas** que compiten constantemente por nuestra atención mediante la **venta de spam** y la **manipulación de nuestras conductas**.

Si el diario en el siglo XIX o la radio y la tele en el siglo XX podían con sus rudimentarios recursos modificar conductas sociales, qué le queda al **monstruoso procesamiento de datos** que realizan Meta, Alphabet (Google), X/Twitter, Amazon y un par más que manejan la información de billones de personas en el globo. Y que, dicho sea de paso, son [las empresas más valiosas del mundo por market cap](#).

Retirarse nunca fue una opción

En la silenciosa era pre Internet podías apagar la radio o la televisión y a otra cosa. En la actualidad, la desconexión parecería ser el sueño del antiguo ermitaño que huye de la sociedad, poéticamente quizás, como Thoreau relata en su *Walden*. Y parte del error radica en pensar que la desconexión es una solución, ya que asociamos irremediablemente nuestra **vida digital** a la **ansiedad infernal** de las redes de los oligarcas del dato, al **peaje constante** del ecosistema de plataformas



de streaming de todo tipo, al **scrolleo infinito** y la **serotonina dosificada** en likes.

Pero nada más aberrante que los delirios místicos de la desconexión cuando somos **personas de Internet**. Pensar en esos términos significa tirar la toalla en [la lucha por la soberanía cognitiva](#): si la solución ante el bombardeo de estímulos y de psyops digitales es huir de la Red de Redes, es porque no escucharon el suficiente punk rock como para plantarse.

Precisamente, desde [Cybercirujas](#) –y fundamentalmente desde las bases del **hacktivismo**– siempre luchamos por ser dueños de nuestros espacios digitales. La llegada de los **smartphones y su ecosistema de apps**, que hacen que las generaciones post 2010 [no conozcan el concepto de file ni de Internet](#), torció la lucha tan rápidamente que recién en los últimos años comenzamos a reaccionar. Los teléfonos inteligentes se instauraron como **cajas completamente oscuras**, de las cuales nada se sabe, nada se pregunta y todo se consume por default.

¿Se imaginan si en sus PCs solo pudieran instalar software a través de la store de Windows? O en las Mac, MacOS. En ese distópico y loco mundo, no existirían Steam ni tantos sitios de descargas. Exactamente eso es lo que sucede con Android y peor aun con iOS: el usuario, o más bien el consumidor –porque su única función es consumir, la noción de uso es completamente secundaria– es naturalmente llevado hacia aquel **ecosistema de la atención** de forma irremediable, lo cual termina a la larga imponiendo ese sentimiento de apatía y desinterés colectivo. Parafraseando a Ricky y Flema: [“Las cosas son así y así”](#)



siempre serán, y aunque vos me jodas, nunca cambiarán”.

Pero, como buenos punks, hemos nacido irremediablemente para joder un poco, y parte de aquel precepto de **Hakim Bay** de las **zonas temporalmente autónomas (TAZ)** tiene que ver con esto. Hay maneras de lograr cierta **autonomía cognitiva** sin irse a vivir a una montaña. Si el teléfono celular es la mayor herramienta de dominación creada y si, aún así, no podemos dejarla –como tampoco podemos renunciar a una vida sin heladera–, al menos podemos lograr un mínimo control de aquel aparato que **atenta contra nuestra independencia cognitiva**. Reducir el email a la función que cumple

Google y los fabricantes de hardware prácticamente dominan nuestras comunicaciones digitales a través de **Android y su ecosistema de aplicaciones**. ¿No hace ruido que prácticamente todo el mundo use Gmail y que resulte extraño mudarse a una nueva cuenta de correo electrónico? El primer paso para luchar por la soberanía cognitiva es **migrar de proveedor de correo electrónico**: recordemos que es una tecnología que existe desde los '70 pero que la **oligarquía tecnológica** ha ido adueñándose-

se y naturalizando ese monopolio a punta de **extractivismo digital**. Existen varios proveedores alternativos serios, desde los más corporativos que ofrecen mail gratuito, como **Protonmail** o **tutanota**, hasta iniciativas más comunitarias como **disroot**, **riseup.net**, **undernet.uy**, entre otros.

Mudar de mail no significa abandonar el Drive de Google sino simplemente hacer una **limpieza mental**, un restart de las comunicaciones hacia zonas más autónomas. Y significa también dejar de usar la aplicación de Gmail: el correo electrónico sirve para enviar y recibir correspondencia digital, **no se precisa de aplicaciones pesadas e invasivas como Gmail**. Hay clientes de correo



electrónico como [Thunderbird](#), que incluso sirven para gestionar cuentas de GMail, si quieren una app para seguir usando esa cuenta que probablemente acarree más de 15 años en infinidad de bases de datos.

La Internet que nos quieren quitar

Situación análoga la del **navegador web**. Chrome tiene más del 65% de cuota de mercado como browser y es responsable de las mismas prácticas que Microsoft impuso en su momento con Internet Explorer, esto es, **direccionar hacia dónde debe ir la Web y aferrarse a los usuarios**, aunque Google lo hace a fuerza de costumbre y de su aceitado ecosistema.

Alternativas a navegadores hay varias, y ya no se trata de switchear a Firefox y quejarse porque no es de nuestro gusto: existen Chromes “de-googleados”, [como Brave o Ungogged Chrome](#), que vienen sin nada del trackeo que Google mete en el navegador. Son esas **falsas features** que la empresa ofrece las que operan psicológicamente junto al resto del aceitado ecosistema de

aplicaciones de redes sociales. Salirse de Chrome y de GMail no implica abandonar Twitter o WhatsApp sino simplemente **dejar de hacer las cosas por default**, que básicamente es lo que explotan los magnates de la economía de la atención.

Tanto triunfaron en el mundo de la computación de mano que lograron imponer una peculiar conducta de mercado: **la sectorización de las tarifas de Internet**. Como sucede con todos los servicios de la era industrial, tradicionalmente se pagaba una boleta de agua, luz o gas y se recibía agua, luz y gas. Con Internet pasaba lo mismo, hasta que [al capitalismo de plataformas se le ocurrió que debíamos volver a pagar Internet](#) para ver películas, escuchar música, bajar libros; es decir, para consumir aquello que ya estaba allí disponible en la red. ¿Se imaginan el quilombo que se armaría si EDENOR, AYSA o GASNEA dijeran: “Bueno, ahora el aire acondicionado, la ducha, la cocina o el inodoro pagan tarifas diferenciadas”? En Internet esto se ha naturalizado, pero nosotros venimos de una escuela que siempre pregonó que **todo lo que está en Internet es tuyo**.

Por eso no sólo es posible sino que es necesario **saltearse todas las putas barreras**



arancelarias de quienes lucran con nuestra atención: con los datos que nos extraen, bien podrían pagarnos por utilizar las aplicaciones, en vez de cobrarnos. No puedo concebir la vida sin [bloqueadores de anuncios](#) para YouTube, tanto en escritorio como en mobile. Incluso las versiones de Chrome para Android degoogleadas, como [Chromite](#), ya traen incorporado un bloqueador. Existen también otras formas de consumir todas esas **redes extractivistas** de nuestra serotonina en nuestros teléfonos celulares.

Liberando al androide

Android, al ser **software libre**, posee mayores posibilidades de plantear la lucha por la soberanía cognitiva. Podemos instalar una **tienda de aplicaciones libres**, que no extraen nuestra data ni lucran con nuestro uso, y acceder a software que está pensado para el usuario y no para un mero consumidor. [F-Droid](#) es el repositorio más grande y seguro de **software libre para Android**.

Allí encontraremos aplicaciones como [Ri-Music](#) (que permite acceder al catálogo de YouTube Music sin loguearse, con funciones de descarga o reproducción en streaming) o [NewPipe](#) y similares (que ofrecen la misma función pero orientada al formato audiovisual de YouTube). No se trata, como digo, de recluirse en la montaña analógica, sino de **usar las redes como queremos**: no voy a dejar de ver YouTube, pero lo voy a hacer **como yo quiero**.

La computadora de escritorio como refugio

Sin dudas, son redes como Twitter o Instagram las que se encargan de bombardearnos con ideas, pensamientos y consumos **que no queremos** pero que, a la larga, por la **exposición constante**, *necesitamos* o deseamos. Parte de ese deseo autoimpuesto entra, por supuesto, a través de la pantalla del celular. Dejar de consumir completamente esas redes puede ser una opción, pero no tod@s tienen por qué aceptarla; ya sea por laburo, pertenencia o FOMO, mucho sucede allí. **Lo ideal es quitarlas de nuestros teléfonos**, erradicarlas de la palma

de la mano y de los bolsillos y consumir las exclusivamente en modo desktop: en la computadora de escritorio se tiene **más control de la atención**.

[La PC, como plataforma de trabajo, de ocio, de vida digital](#), es una herramienta completamente distinta donde la atención se dispersa y maneja de maneras diferentes. Además, entre bloqueadores de anuncios, navegadores sin rastreo y sistemas operativos libres, podemos tener una **higiene cognitiva digital** mucho más controlada que en el mercado de spam de los smartphones. Sentarse en la compu a consumir Internet no es un acto nostálgico de quienes vivimos la era dial-up, sino más bien una decisión que habla de nuestra necesidad de **decidir cómo habitar internet** a través de una autonomía cognitiva alejada de los designios de los tecnócratas del dato.

Si millones de moscas comen mierda...

Finalmente, a la hora de tener **cierta independencia al usar mensajería instantánea**, la cosa está más áspera. Sabido es que los cambios que pone WhatsApp para su plataforma poco tienen que ver con las facilidades y mejoras para comunicarse por lo que otrora conocíamos como “chat”. **Ya nadie habla de chatear porque ese tipo de comunicación murió**.

Cuando uno chateaba no necesitaba tener un **desesperante feedback** que te muestre que la persona está por escribir, por mandar mensaje, ni un patrón de diseño completamente oscuro hecho exclusivamente para que nos quedemos dentro de esa aplicación. WhatsApp deja muchísimo que desear y su uso solamente se explica por el conocido **“efecto de red”**, es decir, vale porque mucha gente lo usa: si millones de moscas comen mierda, la mierda debe ser buena.

Contra ese tipo de manejos resulta sumamente difícil contrariar sin caer en el **ermitañosismo analógico** ya mencionado: abandonar esas plataformas no es una opción. Pero sí saber que existen otras. [No me refiero a Telegram](#), que tiene más problemas y peores prácticas; tampoco pienso en Signal, sino más



bien en algo básico, estandarizado. [XMPP](#) es un protocolo de mensajería instantánea que existe hace más de 25 años y forma parte del stack de Internet, así como [HTTPS](#), el correo electrónico o la [World Wide Web](#).

Anteriormente conocido como [Jabber](#) (utilizado por Google en la era de Google Talk), sigue en desarrollo, tiene clientes mobile para Android y iOS y la mayoría de las veces permite comunicarse entre personas sin utilizar un número de teléfono. No tiene servidores centralizados, permitiendo la **encriptación** de forma sencilla; no recolecta datos y brinda las mismas características que sus competidores: mensajes, audios, vi-

deos, imágenes, llamadas y videollamadas. En [el último número del fanzine cybercirujá](#) publicamos un tutorial de esta maravillosa plataforma de mensajería.

Ante la apatía, la desidia y el consumo por default, optamos por la acción, por decidir qué y cómo habitar la red. Así como deciden leer este sitio desde la web, desde un lector RSS, desde Instagram o Twitter, también pueden optar por luchar en el campo de la **psiquis digital**. No hay que ser un hacker peligroso ni ponerse un sombrero de aluminio: hay que ser más punks y no entregarse de gambas sin siquiera saberlo. No cagaremos al sistema, pero al menos lo intentaremos.



Urbit: la privacidad digital según Curtis Yarvin

Revisamos los elementos
claves del proyecto de Tlon
Corporation, que busca corregir
la falta de privacidad y el exceso
de centralización digital.



x Giancarlo Brusca



¿Qué pasaría si fuéramos dueños de toda nuestra **vida digital** sin tener que hacer esfuerzos extra para lograrlo? Esa es la promesa de **Urbit**, un proyecto creado por **Curtis Yarvin** en 2002, que desde entonces se viene cocinando lentamente en las sombras. Así como crypto es el “parche” de internet que viene a corregir el problema de la centralización y falta de privacidad en las transacciones de información, **Urbit** apunta a corregir el stack entero.



Urbit

Urbit es el proyecto principal de la **Tlon Corporation**, que lleva su nombre en honor al cuento **Tlon, Uqbar, Orbis Tertius, de Jorge Luis Borges**. Es un Sistema Operativo Virtual –necesita de un SO subyacente para existir, como Windows, Linux o MacOS– hecho de cero, completamente descentralizado y privado. Cada computadora recibe el nombre de “**nave**” y funciona como servidor propio, todas las **naves** se conectan entre sí a través de una **red peer-to-peer (P2P)** llamada **Ames**. Para identificarse dentro del sistema, cada nave tiene un ID propio que es un NFT dentro de la red **Ethereum**; y este modelo de identidad –diseñado también por y para **Urbit**– se llama **Azimuth**.

La filosofía y razón de ser de **Urbit** se basa en que toda nuestra **vida digital** está construida sobre cimientos que apuntan a que el usuario no tenga control total sobre sus datos. Todos usamos sistemas operativos que tienden a la centralización o dependen de conocimientos técnicos muy altos y el uso de herramientas externas complejas para llegar a tener privacidad real (como **Tails**, **Qubes OS** o algunas dis-

tribuciones de Linux). Para cumplir ese objetivo, **Curtis Yarvin** decidió reinventar la rueda y, en 2002, comenzó creando **Nock**.

Nock

Nock, según cuenta la leyenda, fue programado enteramente por un joven **Yarvin**, y es la base de todo el sistema. Es un **lenguaje de programación de bajo nivel**, completamente nuevo, definido en una especificación de ~33 líneas de código que describe operaciones funcionales. Estas operaciones son interpretadas por **Vere**, el entorno de ejecución (o runtime) de **Urbit** escrito en lenguaje C.

Vere

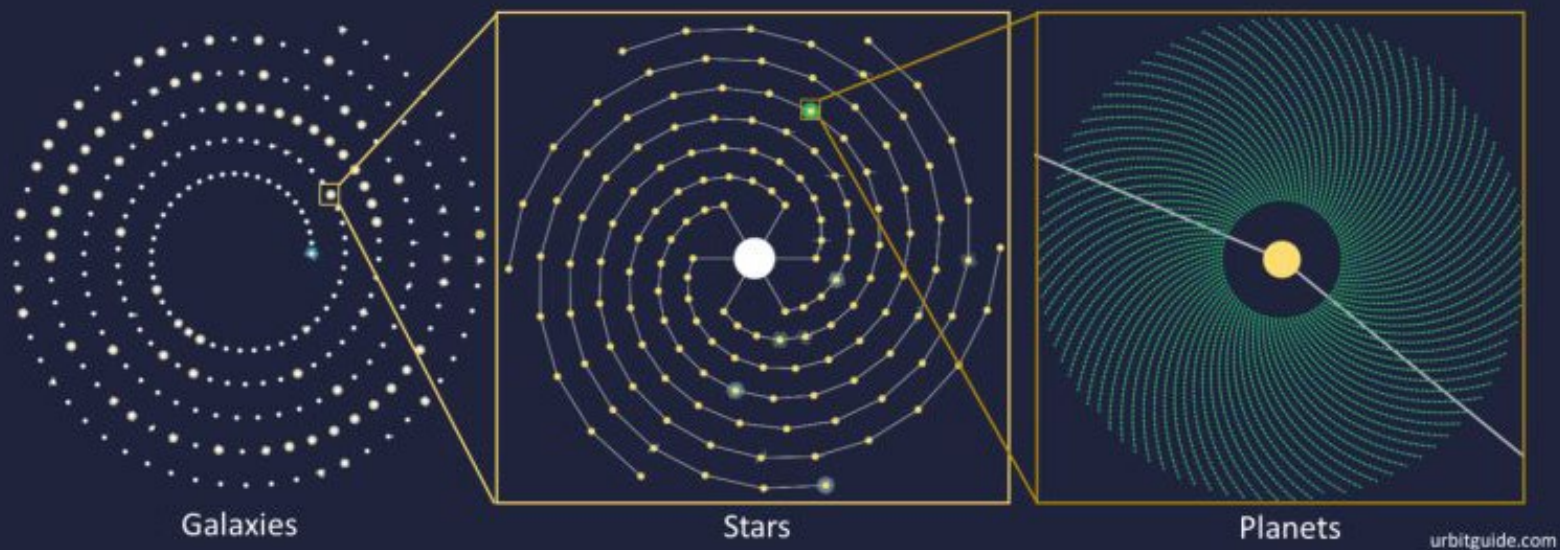
Vere ejecuta el código, implementando las operaciones de **Nock** en el sistema operativo subyacente (Windows, Linux, MacOS). Todo lenguaje de bajo nivel como **Nock** se complementa con un **lenguaje de alto nivel** más simple, para que los seres humanos normales como nosotros podamos contribuir al sistema y codear nuestras aplicaciones. El lenguaje de alto nivel de **Urbit** se llama **Hoon**. En resumen: el lenguaje de alto nivel (**Hoon**) se compila a lenguaje de bajo nivel (**Nock**) y el entorno de ejecución (**Vere**) lo ejecuta.

Hoon

Hoon es, entonces, el lenguaje de alto nivel en el que está construido el mayor porcentaje de **Urbit**. Es un lenguaje funcional también creado de cero, bastante complejo (pero menos que **Nock**) y en el que se realiza el 99% de contribuciones de la comunidad. Hay contribuciones a **Nock** pero son las menos: de hecho, la última fue en 2016.

Arvo / Urbit OS

En **Hoon** están escritas todas las aplicaciones, así como también la red **Ames** y el sistema operativo propiamente dicho, que se llama **Arvo** (o **Urbit OS**), encargado de gestionar



eventos, ejecutar aplicaciones, administrar el estado de la “nave”, manejar la interfaz y coordinar los módulos que se encargan de funcionalidades específicas e importantes del sistema (así como **Ames** es la red P2P, existen otros como **Clay**, por ejemplo, que se encarga del sistema de archivos -*filesystem*).

Naves

Cada **nave** ejecuta su propia copia de **Arvo** y almacena sus datos (mensajes, archivos, aplicaciones) localmente, sin depender de un servidor centralizado como en aplicaciones tradicionales (WhatsApp, Gmail). Vos sos el dueño de tu **nave**, tus datos y tu identidad. Tu **nave** puede servir contenido (como publicar posts, alojar aplicaciones o compartir archivos) directamente a otras a través de la red **Ames**.

Cómo correr Urbit localmente

Ejemplo práctico de lo que podés hacer con esto: instalás **Urbit** en tu computadora y creás una nave llamada *~sampil-palnet* (los IDs de **Azimuth** tienen estos nombres). Tu computadora corre **Vere**, que ejecuta **Arvo** en tu nave. Usás una aplicación de chat y le enviás un mensaje a otra persona (por ejemplo, *~dolpan-lacfel*), tu nave lo cifra y lo envía directamente a esa otra vía **Ames**, sin pasar por un servidor central.

De hecho, yo hice exactamente esto. La comunidad de **Urbit** no es tan grande como la de **Remilia** aún, pero son amigables y no es difícil encontrarlos en X si sabés buscar. Así fue como contacté a [без телеологии \(@sitful_hatred\)](#) y le comenté que tenía ganas de escribir esta nota. Su respuesta inmediata fue la de regalarme un ID para que pueda ingresar a la versión oficial de **Urbit** (los ID, al ser de Ethereum, cuestan plata). Me lo envió por mail y procedí a correr **Urbit** localmente.

Correr el sistema localmente todavía no es fácil para gente que nunca usó la terminal, pero [acá tienen el tutorial oficial por si lo quieren intentar](#). **Tlon Messenger** viene por defecto y es la aplicación de chat oficial de **Urbit**, es una especie de Telegram pero más raro y, como todo en este universo, 100% privado y descentralizado. Además de chats 1-1, hay bastantes grupos clasificados por temas para poder interactuar con la comunidad.

Conclusión

Mi experiencia general usando **Urbit** fue la de estar probando algo que está en una etapa bastante inicial de su desarrollo. Hay una infinidad de trabajo por delante, pero el progreso se está haciendo y la comunidad es fuerte. Sin dudas un proyecto al que vale la pena mirar de reojo cada tanto a ver en qué anda. Y, por qué no, contribuir si te interesa.



421

FOR EDUCATION

421
BROADCASTING &
NETWORK

YOU WANT THE BEST? YOU WANT THE BEST? YOU WANT THE BEST?

THE



cuatroveintiuno.com

